

Tecnología Digital VI

Modelos Lineales

Licenciatura en Tecnología Digital - UTDT

Regresión Lineal

En Inferencia Estadística ya vieron este modelo.

No solo conocieron el modelo sino también la métrica para optimizarlo, ¿cuál era?

También vieron el algoritmo para esa optimización, ¿cuál era?

The diagram shows the linear regression equation $Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$ with the following labels and annotations:

- Dependent Variable**: Points to Y_i .
- Population Y intercept**: Points to β_0 .
- Population Slope Coefficient**: Points to β_1 .
- Independent Variable**: Points to X_i .
- Random Error term**: Points to ϵ_i .
- Linear component**: A bracket under $\beta_0 + \beta_1 X_i$.
- Random Error component**: A bracket under ϵ_i .

Regresión Lineal – sklearn

Usamos la librería Scikit-learn para entrenar nuestro primer modelo.

Utilizamos la implementación de:

- Dataset:
https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset
- El modelo propiamente dicho.
- `train_test_split` para partir los datos.
- `Mean_squared_error` para la métrica de evaluación.

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load the California Housing dataset
dataset = fetch_california_housing()
X, y = dataset.data, dataset.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Create an instance of the LinearRegression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

Mean Squared Error: 0.5289841670367247
```

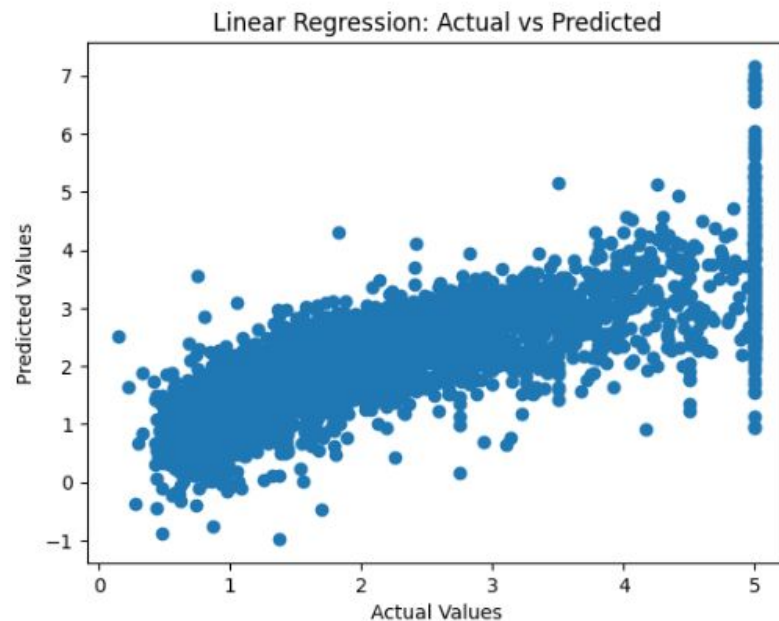
Regresión - sklearn

Si la regresión lineal fuese perfecta, veríamos una recta ideal entre los valores reales y los inferidos.

En este caso tenemos una buena correlación positiva, pero con algunos problemas.

```
import matplotlib.pyplot as plt

# Create a scatter plot of predicted vs actual values
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()
```



Regresión Lineal (que no es una línea)

Se llama regresión lineal porque es lineal en la combinación de sus parámetros, pero uno de sus parámetros puede ser una columna elevada al cuadrado. De esta manera (y de varias más sofisticadas) es que podemos escaparnos de la linealidad.

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load the California Housing dataset
dataset = fetch_california_housing()
X, y = dataset.data, dataset.target

# Square column 3 and add them as new features
new_features = np.multiply(X[:, 3], X[:, 3])
X_with_new_features = np.column_stack((X, new_features))

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_with_new_features, y, test_size=0.2, random_state=0)

# Create an instance of the LinearRegression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

Mean Squared Error: 0.5268740576092612
```

Regresión Lineal

En este ejemplo vemos una función cúbica con algo de ruido y luego intentamos ajustarla con una línea recta.

Capturamos la tendencia pero estamos lejos de los puntos.

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Generate the dataset
np.random.seed(42) # For reproducibility

# Define the cubic function
def cubic_function(x):
    return 3 * x**3 - 2 * x**2 + 5 * x + 10

# Generate x values
x = np.linspace(-5, 5, 100)

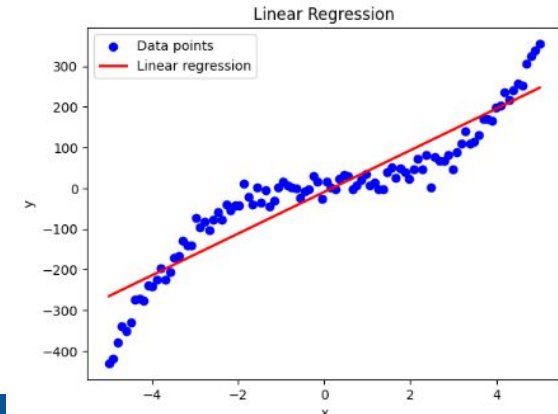
# Generate y values with some noise
y = cubic_function(x) + np.random.normal(0, 20, x.shape)

# Reshape the x array to match the expected input of the linear regression model
x = x.reshape(-1, 1)

# Fit the linear regression model
regressor = LinearRegression()
regressor.fit(x, y)

# Predict the y values using the fitted model
y_pred = regressor.predict(x)

# Plot the data points and the regression line
plt.scatter(x, y, color='blue', label='Data points')
plt.plot(x, y_pred, color='red', linewidth=2, label='Linear regression')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear Regression')
plt.legend()
plt.show()
```



Regresión Lineal

Haciendo Feature Engineering y agregando dos nuevas columnas (el cuadrado y el cubo de la original) podemos ajustar muy bien los datos aún utilizando una regresión lineal.

```
# Reshape the x array to match the expected input of the linear regression model
x = x.reshape(-1, 1)

# Create additional features (x^2 and x^3)
x_square = x**2
x_cube = x**3

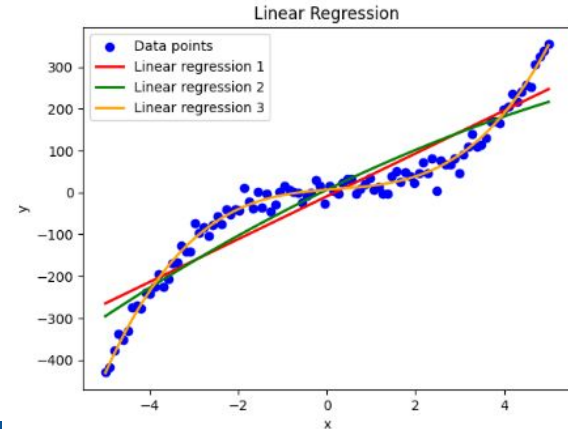
# Fit the linear regression models
regressor1 = LinearRegression()
regressor1.fit(x, y)

regressor2 = LinearRegression()
regressor2.fit(np.hstack((x, x_square)), y)

regressor3 = LinearRegression()
regressor3.fit(np.hstack((x, x_square, x_cube)), y)

# Predict the y values using the fitted models
y_pred1 = regressor1.predict(x)
y_pred2 = regressor2.predict(np.hstack((x, x_square)))
y_pred3 = regressor3.predict(np.hstack((x, x_square, x_cube)))

# Plot the data points and the regression lines
plt.scatter(x, y, color='blue', label='Data points')
plt.plot(x, y_pred1, color='red', linewidth=2, label='Linear regression 1')
plt.plot(x, y_pred2, color='green', linewidth=2, label='Linear regression 2')
plt.plot(x, y_pred3, color='orange', linewidth=2, label='Linear regression 3')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear Regression')
plt.legend()
plt.show()
```



Regresión Logística

La regresión lineal es el modelo lineal canónico para el problema de regresión. Su contraparte clásica en el problema de clasificación es, paradójicamente, el modelo de Regresión Logística.

Formalmente, y como indica su nombre, una Regresión Logística también es un tipo de regresión pero se utiliza de manera canónica en los problemas de clasificación binaria mediante la utilización de un threshold como paso posterior de haber realizado la regresión propiamente dicha.

Regresión Logística

Las *odds* o *chances*, no son más que una proporción de probabilidades. Es la probabilidad de que algo sucede dividido la probabilidad de que no suceda.

La función *logit* de una probabilidad es tomar el logaritmo de las chances. Lo que se conoce como *log-odds*.

$$Odds = \frac{p}{1 - p}$$

$$logit(p) = \ln(Odds) = \ln \left(\frac{p}{1 - p} \right)$$

Regresión Logística

$$\log \frac{p}{(1-p)} = \text{logit}(p) = w_1x_1 + \dots + w_mx_m + b = \sum w_jx_j + b = \mathbf{w}^T \mathbf{x} + b$$

Así como en una regresión lineal *asumimos* una relación directa entre la columna objetivo y una combinación lineal del resto de las columnas, en una Regresión Logística *asumimos* una relación lineal entre los features y los *log-odds*.

Regresión Logística

Exponenciado a ambos lados de la igualdad y luego haciendo los despejes necesarios podemos modelar la probabilidad de pertenecer a la clase positiva (estamos en una clasificación binaria por el momento) como una función de las columnas de entrada.

La función σ (sigma) se denomina función *sigmoidea* o logística.

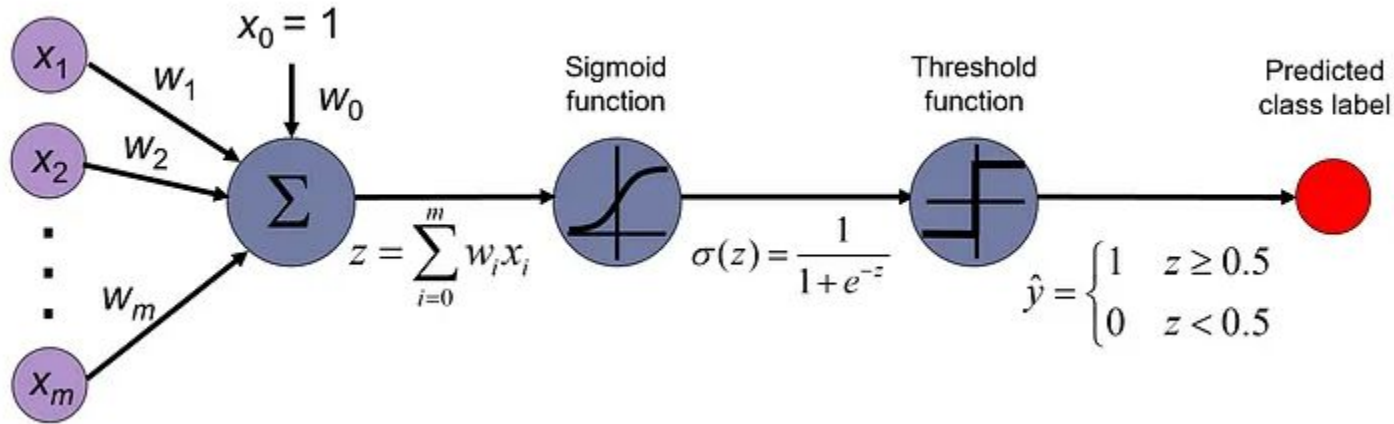
$$\frac{p}{1-p} = e^{\mathbf{w}^T \mathbf{x}}$$

$$p = (1-p)e^{\mathbf{w}^T \mathbf{x}}$$

$$p(1 + e^{\mathbf{w}^T \mathbf{x}}) = e^{\mathbf{w}^T \mathbf{x}}$$

$$p = \frac{e^{\mathbf{w}^T \mathbf{x}}}{(1 + e^{\mathbf{w}^T \mathbf{x}})} = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} = \sigma(\mathbf{w}^T \mathbf{x})$$

Regresión Logística – Esquema



Podemos pensar todo el modelo como los siguientes pasos: 1) combinar linealmente los inputs (junto con un bias) 2) aplicar la función sigmoidea 3) traducir las probabilidades obtenidas a una función indicadora de pertenencia de clase.

Regresión Logística – Loss Function

¿Cómo consigo los pesos $\mathbf{w}$? Necesito una loss function o cost function que, dado un dataset de entrenamiento, encuentre los pesos de este modelo.

Lo que vamos a maximizar es la función de likelihood o verosimilitud. Para esto vamos a hacer la asunción que las muestras son independientes (¿y si no se cumple la asunción?).

$$L(\mathbf{w}) = P(\mathbf{y} \mid \mathbf{x}; \mathbf{w}) = \prod_{i=1}^n P(y^{(i)} \mid x^{(i)}; \mathbf{w}) = \prod_{i=1}^n \left(\phi(\underbrace{z^{(i)}}_{z=\mathbf{w}^T \mathbf{x}}) \right)^{y^{(i)}} \left(1 - \phi(\underbrace{z^{(i)}}_{z=\mathbf{w}^T \mathbf{x}}) \right)^{1-y^{(i)}}$$

Regresión Logística – Loss Function

Tomando logaritmo a ambos lados (nos sirve para el método de optimización):

$$l(\mathbf{w}) = \log L(\mathbf{w}) = \sum_{i=1}^n \left[y^{(i)} \log(\phi(z^{(i)})) + (1 - y^{(i)}) \log(1 - \phi(z^{(i)})) \right]$$

En general las loss functions las pensamos para minimizar así que hacemos negativos ambos lados:

$$J(\mathbf{w}) = \sum_{i=1}^n \left[-y^{(i)} \log(\phi(z^{(i)})) - (1 - y^{(i)}) \log(1 - \phi(z^{(i)})) \right]$$

Regresión Logística – Algoritmo de optimización

En regresión lineal tenemos una “fórmula cerrada” para calcular los pesos del modelo que minimizan la loss function (cuadrados mínimos).

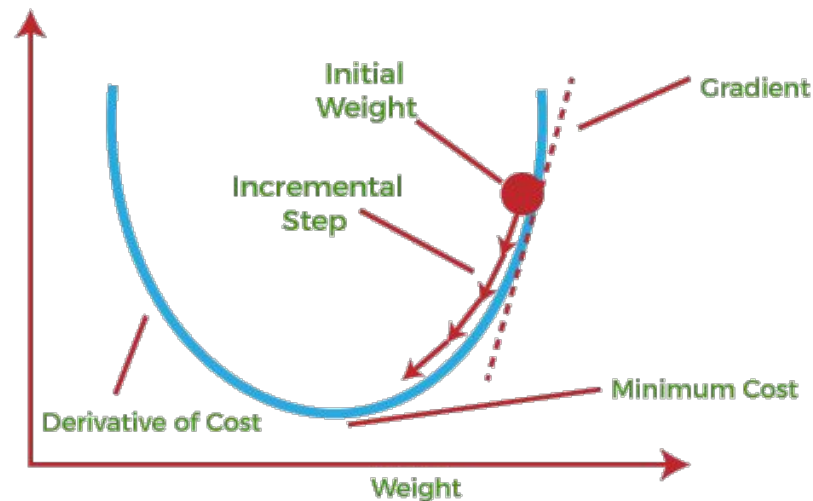
La loss function de la regresión logística (denominada Negative Log-Likelihood o NLL) no tiene un método exacto así que se ataca con métodos iterativos.

EL método iterativo por excelencia en este tipo de modelos de Machine Learning es Gradient Descent (Gradiente Descendiente). De todas maneras, en el caso particular de la regresión logística existen otros métodos iterativos que son preferibles (por velocidad de convergencia y estabilidad).

Gradient Descent

Lo que estamos tratando de hacer es minimizar una función en donde las variables son “los w ”, los parámetros de nuestro modelo.

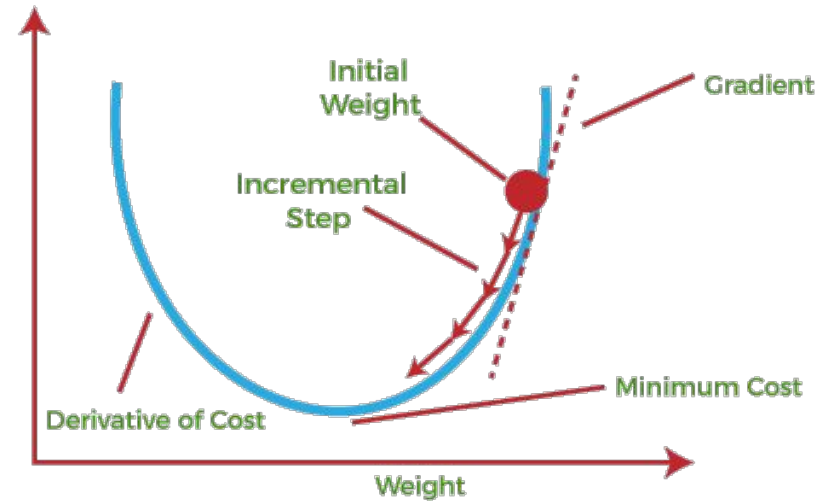
Lo que podemos hacer es empezar con cualquier conjunto de w y luego ir buscando dar pasos en dirección al mínimo de la función.



Gradient Descent

El gradiente de una función es el vector de las derivadas parciales con respecto a cada variable (en este caso, los w_i).

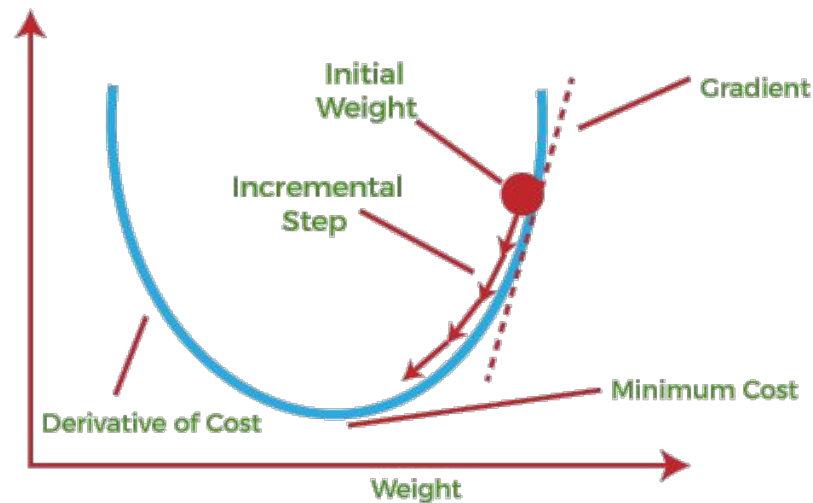
El gradiente en un punto nos indica la dirección de mayor crecimiento de la función. Es por esto que si queremos minimizar una función estando parados en un punto, tiene sentido moverse en la dirección opuesta al gradiente.



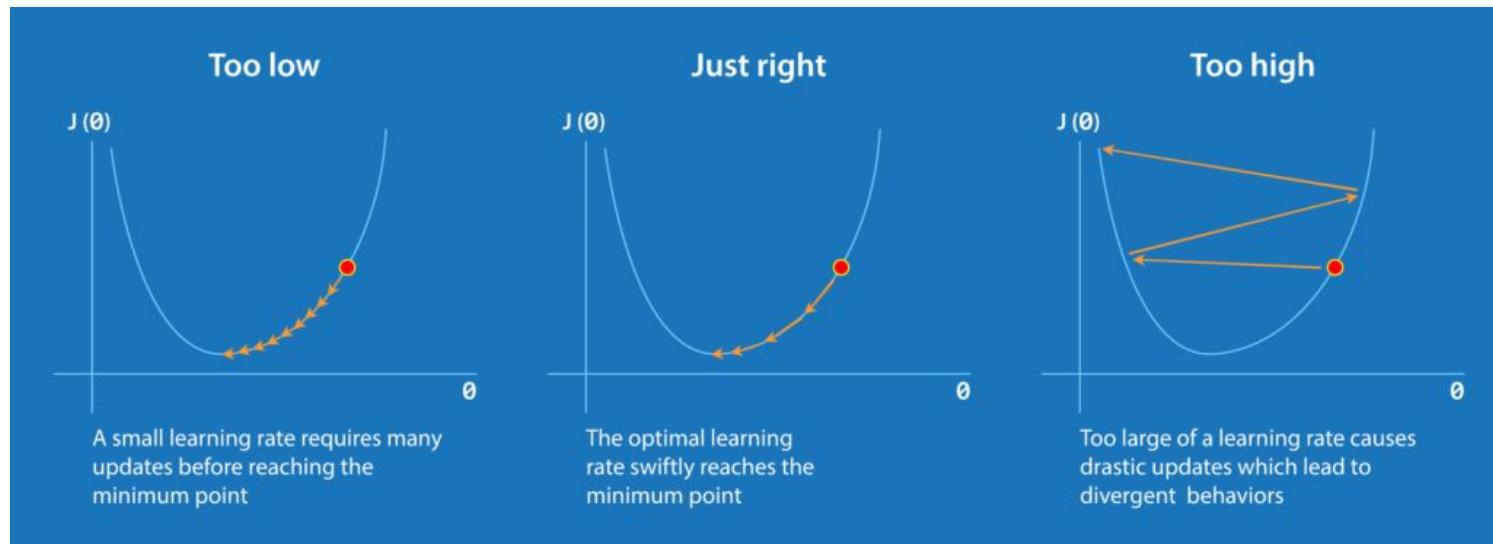
Gradient Descent

En el caso de una función convexa, ir en esta dirección nos asegura estar cayendo al mínimo global si caminamos *la cantidad justa*.

Esa cantidad justa es lo que se conoce como *learning rate* porque el gradiente nos dice la dirección pero nadie nos dice cuánto tenemos que movernos en esa dirección.



Gradient Descent – Learning Rate



Con un learning rate muy bajo, corremos el riesgo de converger muy lentamente.

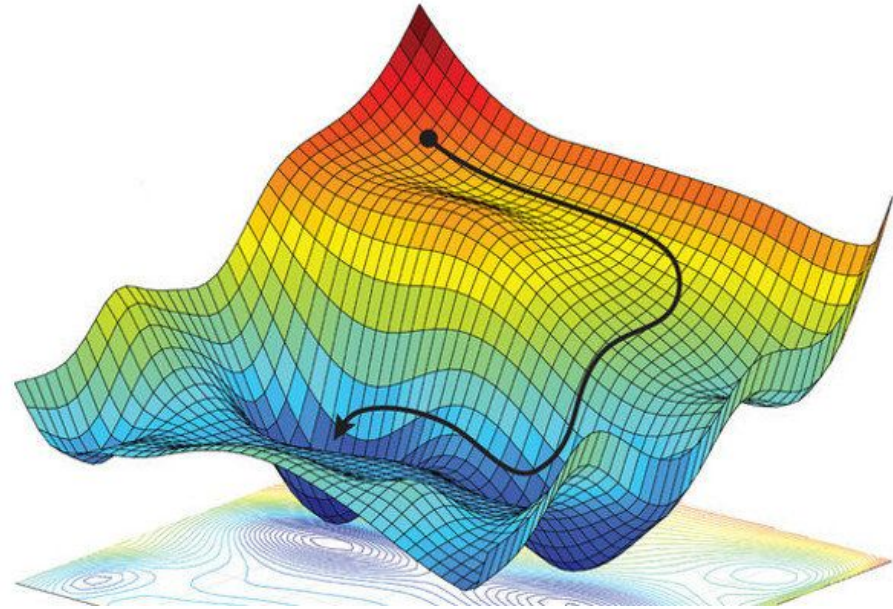
Con un learning rate muy alto, corremos el riesgo de pasarnos de largo.

Gradient Descent – Non-Convex

¿Qué pasa si la función no es convexa?

GD podría converger un mínimo local.

No es necesariamente un problema porque igualmente es un mínimo local y puede servir, pero nos podemos perder de explorar otras regiones.



Más allá de Gradient Descent

GD es el algoritmo base para este tipo de optimización, pero luego hay varias cuestiones adicionales que en la práctica son muy útiles:

- SGD (Stochastic Gradient Descent): no tener que computar todo el gradiente si no ir haciendo una aproximación. Más rápido y aparte suma *ruido* que nos puede sacar de mínimos locales.
- Introducción de la noción de *momentum* para ir acumulando gradientes.
- Incorporación de Learning Rates adaptativos (ADAM) para que vaya variando durante la optimización.

Regresión Logística – sklearn

Scikit-learn tiene una implementación para Regresión Logística

(https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)

Lo aplicamos out-of-the-box para el dataset de breast cancer

([https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)))

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load the breast cancer dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target variable (0 - Benign, 1 - Malignant)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a logistic regression model
model = LogisticRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

Accuracy: 0.956140350877193
```

Gradient Descent *a mano*

La clase LogisticRegression de sklearn ya tiene sus optimizadores implementados adentro.

¿Qué necesitamos si queremos hacer nosotros el solver?

- Las derivadas parciales de la función de costo.
- Un learning rate para hacer un paso de la actualización de los pesos.

Let's start by calculating the partial derivative of the log-likelihood function with respect to the j th weight:

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z)$$

Before we continue, let's also calculate the partial derivative of the sigmoid function:

$$\frac{\partial}{\partial z} \phi(z) = \frac{\partial}{\partial z} \frac{1}{1+e^{-z}} = \frac{1}{(1+e^{-z})^2} e^{-z} = \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right) = \phi(z)(1-\phi(z))$$

Now, we can resubstitute $\frac{\partial}{\partial z} \phi(z) = \phi(z)(1-\phi(z))$ in our first equation to obtain the following:

$$\begin{aligned} \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z) &= \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \phi(z)(1-\phi(z)) \frac{\partial}{\partial w_j} z \\ &= \left(y(1-\phi(z)) - (1-y)\phi(z) \right) x_j \\ &= (y - \phi(z)) x_j \end{aligned}$$

Remember that the goal is to find the weights that maximize the log-likelihood so that we perform the update for each weight as follows:

$$w_j := w_j + \eta \sum_{i=1}^n \left(y^{(i)} - \phi(z^{(i)}) \right) x_j^{(i)}$$

Gradient Descent *a mano*

Implementamos la función sigmoidea.

Luego implementamos GD haciendo un número fijo de iteraciones. En cada paso:

- Calculamos $\hat{y}$.
- Calculamos el gradiente.
- Realizamos la actualización de los pesos del modelo.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the breast cancer dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target variable (0 - Benign, 1 - Malignant)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Add a column of ones to the feature matrix for the bias term
X_train = np.hstack((np.ones((X_train.shape[0], 1)), X_train))
X_test = np.hstack((np.ones((X_test.shape[0], 1)), X_test))

# Initialize the weights
np.random.seed(42)
num_features = X_train.shape[1]
weights = np.random.randn(num_features)

# Define the sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Define the gradient descent function
def gradient_descent(X, y, weights, learning_rate, num_iterations):
    for i in range(num_iterations):
        y_pred = sigmoid(np.dot(X, weights))
        gradient = np.dot(X.T, (y_pred - y))
        weights -= learning_rate * gradient

    return weights
```


Gradient Descent *a mano*

Con un *learning rate* y cantidad de iteraciones adecuado, conseguimos una performance similar a la obtenida usando sklearn.

```
# Set hyperparameters
learning_rate = 0.001
num_iterations = 100000

# Perform gradient descent
weights = gradient_descent(X_train, y_train, weights, learning_rate, num_iterations)

# Make predictions on the test set
y_pred = np.round(sigmoid(np.dot(X_test, weights)))

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

Accuracy: 0.9649122807017544
```

Learning Rate bajo

Si el learning rate es demasiado chico, corremos el riesgo de dar pasos muy pequeños y en consecuencia necesitaríamos muchas iteraciones para llegar a un buen valor.

```
# Set hyperparameters
learning_rate = 0.0000001
num_iterations = 100000

# Perform gradient descent
weights = gradient_descent(X_train, y_train, weights, learning_rate, num_iterations)

# Make predictions on the test set
y_pred = np.round(sigmoid(np.dot(X_test, weights)))

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Accuracy: 0.9385964912280702
```

Learning Rate alto

Con un learning rate muy alto, corremos el riesgo de rebotar y que tampoco terminemos consiguiendo un buen valor final.

```
# Set hyperparameters
learning_rate = 1
num_iterations = 100000

# Perform gradient descent
weights = gradient_descent(X_train, y_train, weights, learning_rate, num_iterations)

# Make predictions on the test set
y_pred = np.round(sigmoid(np.dot(X_test, weights)))

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Mejorando Logistic Regression

En el caso en el que usamos Scikit-learn, lo hicimos completamente *out-of-the-box*. No hicimos ningún preprocesamiento del dataset y tampoco utilizamos ninguno de los varios parámetros que provee el modelo.

Hay muchas posibilidades que vimos en el camino de ML, vamos a utilizar dos:

- Feature Transformation: vamos a escalar las columnas.
- Feature Selection: vamos a utilizar regularización.

Feature Scaling

La Regresión Logística es un modelo que asume una relación lineal entre las columnas y las *log-odds*. Si los features están en escalas muy diferentes, entonces esa combinación lineal estará muy desbalanceada desde un principio y los pesos tendrán que ajustarse mucho para compensar esto.

Vamos a utilizar la clase `StandardScaler` de Scikit-learn (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>)

Como regla intuitiva, si la loss function de mi modelo depende de algún tipo de distancia, posiblemente sirva hacer feature scaling.

Regularización

Otro paso que podemos implementar en Regresión Logística para buscar mejores resultados es aplicar la regularización de Lasso o Ridge en la función de costo.

Estas nuevas funciones de costo pueden ayudar a evitar el overfitting.

$$J_{Lasso}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=1}^m |w_j|$$

$$J_{Ridge}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=1}^m w_j^2$$

Regularización

Regula la libertad del modelo

La idea de las técnicas de regularización es prevenir que los parámetros del modelo tengan tanta libertad que sobreajusten al set de entrenamiento.

Ambas técnicas responden a una “función de presupuesto” que penalizan la norma del vector de parámetros. En el caso de Lasso, la norma 1, en el caso de Ridge, la norma 2.

Por cómo funciona cada fórmula, Lasso suele tener la habilidad de directamente anular features, mientras que Ridge puede penalizar fuertemente algunas columnas pero sin desaparecer el coeficiente.

Las técnicas de regularización son **MUY** importantes en la práctica para conseguir modelos que generalicen bien.

Regularización

Para utilizar regularización en regresión logística necesitamos dos parámetros:

- Penalty: indica si queremos l1(Lasso), l2(Ridge), elasticnet(mezcla).
- C: es un coeficiente inverso de regularización, cuanto más chico, más penalización.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load the breast cancer dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target variable (0 - Benign, 1 - Malignant)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create a logistic regression model with L2 regularization
model = LogisticRegression(penalty='l2', C=0.1)

# Fit the model to the training data
model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.9824561403508771

¿Y el gradiente?

Ojo que meter términos a la loss function no es inofensivo al algoritmo de optimización. Con nuevos términos el gradiente de la función cambia.

En particular con el término que se agrega en Lasso la función deja de ser diferenciable.

```
# Create a logistic regression model with L1 regularization
model = LogisticRegression(penalty='l1', C=0.1)

# Fit the model to the training data
model.fit(X_train_scaled, y_train)
```

ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.

Modelo ¿Lineal?

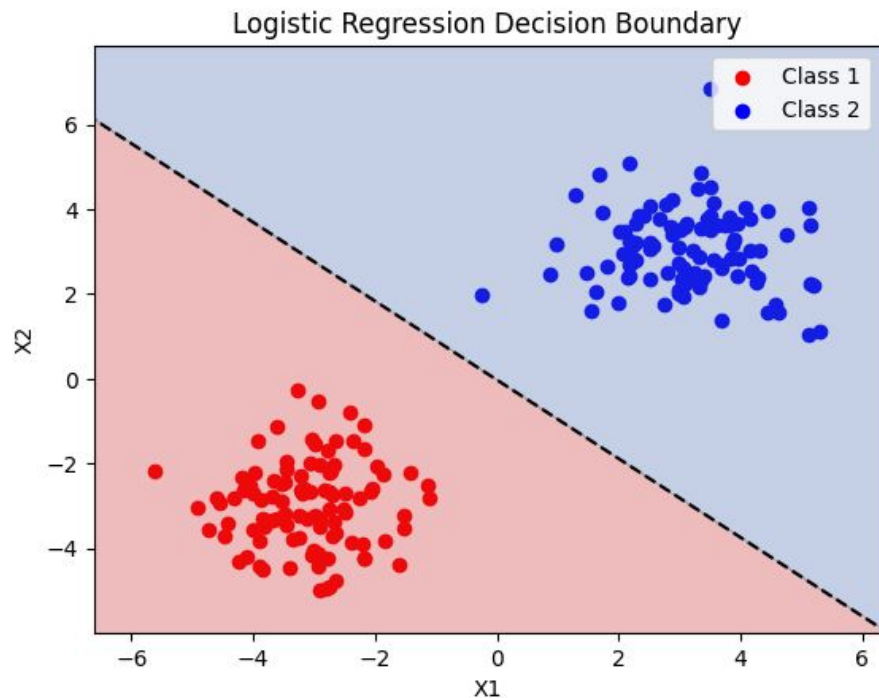
En el modelo de regresión logística aplicamos funciones claramente no lineales para llegar a la clasificación lineal. ¿Por qué entonces decimos que es un modelo lineal?

- La linealidad se da en la relación entre los features y la función logit de la probabilidad (o las log-odds).
- La barrera de pertenencia entre las clases se da **cuando la probabilidad es 0.5**. En ese punto la función logit es 0. Esto quiere decir que la decisión boundary del modelo es exactamente cuando la combinación lineal de los features da 0, lo cual es una hiperplano (una “línea”).

Lo que separa la clase positiva de la clase negativa es si z (la combinación lineal de los features) es igual a cero. Eso hace que el resultado de la función sigmoide de 0,5.

Decision Boundary

Un ejemplo de la decision boundary encontrada por el modelo de regresión logística al ser aplicado en dos nubes de puntos linealmente separables.



Más allá de los modelos lineales

¿Qué sigue después (en esta misma dirección) de los modelos lineales? Los modelos aditivos.

Los modelos aditivos introducen diferentes funciones a los features para obtener modelos más complejos pero sin perder la linealidad.

En el capítulo 7 de *An Introduction To Statistical Learning* se puede encontrar una introducción a diferentes modelos aditivos, finalizando con el modelo GAM (Generalized Additive Model). Los autores del libro son los que desarrollaron estos modelos originalmente.